GentleVanguard

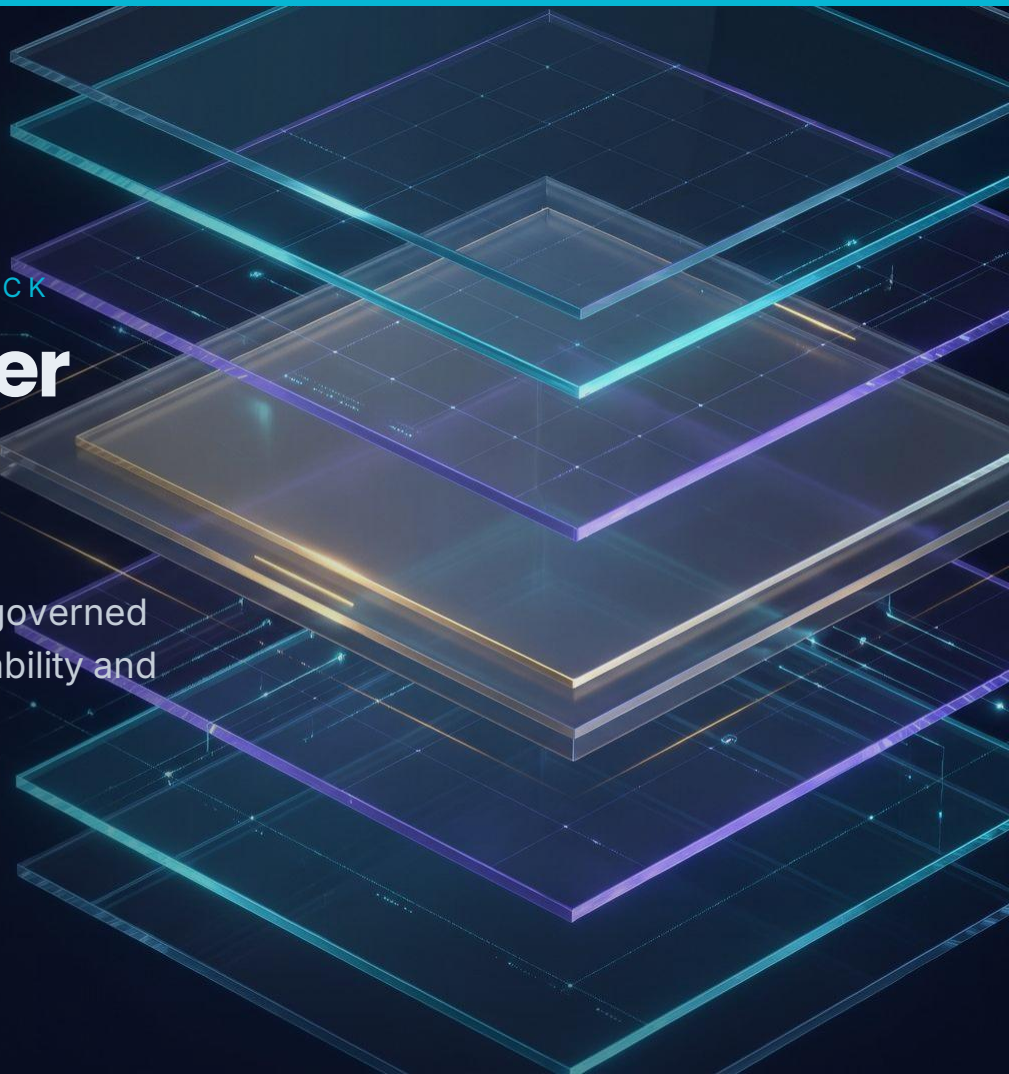
INTERNAL AI ADOPTION - A GOVERNED STACK

An orchestration layer for your company AI

A local-first stack that turns AI assistants into governed capability: defined roles, token budgets, traceability and audit.

Local first | Traceable | Portable

Stack v4.3.0 - November 2026



WHO BUILDS IT

The person who knows applied AI in large companies has done it before

Emmanuel Ortiz Gutierrez: software architect, AI integration engineer and founder of GentleVanguard. More than 15 years building and analysing systems for companies: software architecture and AI integration at DIRECTV Latin America, technical consulting at GlobalLogic and Oracle, plus analysis, QA and development team coordination.

DIRECTV

Latin America

Software architecture and AI integration

GlobalLogic

Oracle

Technical consulting for enterprise clients

UNSJ

Information Systems

BSc in Information Systems

The stack is not a chatbot: it is the layer that governs it

Each layer solves one class of problems and keeps a record of what it did.

A roster of agents

21 agents with model, permissions, step limits and budget assigned.

Capabilities

250 reusable skills invoked by context, not by code.

Evidence

Sessions, traces, events and hash-chained audit.

Controls

Fail-closed policies, secret scanning and permission traceability.

Tools	CLI, IDEs and product surfaces
Agents	Specialized profiles per runtime
Orchestration	Context, routing and steps
Knowledge	Engram, CodeGraph and Graphify
Operational data	Nexus / SQLite, kept separate
Quality & delivery	Gates, review and evidence

Nine layers, each with an explicit boundary

The structure is what makes it possible to audit, replace or extend one piece without breaking the rest.

Surfaces	Ten audited tool profiles (OpenCode, Codex, ZCode, MiniMax Code, Claude Code, Antigravity, Cursor, Cline, Windsurf and VS Code) over one workspace.
Orchestration	Model and agent routing, step escalation by complexity, and a learnable routing table with 17 domains and overrides.
Agents	21 definitions with permissions and step limits; 27 mapped roles and 30 model bindings by domain and criticality.
Capabilities	250 local skills and 32 OpenCode skills, with 28 critical ones under a contract verified in real execution.
Knowledge	Engram (memory), CodeGraph (incremental index), graphify (deterministic AST graph) and Nexus (43 tables, 22 migrations, WAL).
Tools	Ten MCP servers with uniform access, including two external ones behind a circuit breaker.
Infrastructure	Watchtower with 154 checks over 29 components, six operating modes and auto-heal.
Observability	Dashboard with real traces, a 7-component health API, OTLP export and configurable alerts.
Applications	28 of our own apps running on the stack, validating the architecture in production.

SCALE

Not a prototype: a system under sustained use

Figures verified against the repository at version 4.3.0.

786

TypeScript files in src/

Own stack code, organised by area

272

Test files

Unit, e2e, integration, security and skills

583

npm scripts

Every stack operation is an auditable command

154

Health checks

Across 29 components, with auto-heal

CAPABILITIES

What the company gets when it installs the stack

Roles, not people

27 mapped roles with primary and fallback subagents, triggers and skills. AI stops being one person tool and becomes a role with a standard.

Cost under control

A daily budget of 10 million tokens, 3 million per session and 100 thousand per agent, with routing by cost, latency and load.

Explicit permissions

Per-agent permissions, fail-closed policies, an MCP security gateway, secret scanning and a separation between reconstructable and non-reconstructable data.

OPERATION

How a task actually runs, end to end

The loop does not change between a one-week project and a six-month one.

Pre-process

Every turn injects context, skills and rules before the model sees the message.

>

Routing

The router picks model and agent by domain, complexity and cost.

>

Execution

The agent works with real tools and leaves a trace of every step.

>

Verification

Gates and the reviewer close the loop before work is declared done.

GOVERNANCE

What keeps this from being an uncontrolled experiment

30 quality gates

13 on pre-commit, 2 on commit-msg, 2 on post-commit, 3 on pre-push, 8 in CI, 1 governance and 1 on deploy. 28 of 30 enforced strictly.

Supply traceability

Generated and validated SBOM, artifact provenance and a documented mapping against the OWASP Top 10 for agents.

Secret scanning

80 patterns with optional entropy filtering and redaction by default, wired into the hook and the watchtower. Three independent scanners coexist.

Real observability

A dashboard with no mock data: metrics, traces, alerts and feedback come from the operation, not from a mockup.

Chained audit

Event sourcing with prevHash and a SHA-256 hash per event. It detects accidental mutation or inconsistent log editing.

Backup and recovery

Scheduled daily database backup keeping 14 copies, plus recovery drills with measured RTO and RPO.

THE DECISION

Adopting AI tool by tool, or with an orchestration layer

This is the difference between a software contract and an organisational capability.

Governed stack

The capability stays inside the company

- One standard for every team, defined in the repository.
- Budget, permissions and limits per role, not per tool vendor.
- Evidence for every task: traces, events and chained audit.
- Process knowledge in the company database, not in a chat memory.
- Continuity: the stack is upgraded and migrated with versioning.
- Works with whichever model provider the company picks, or with local models.

Tool by tool

Each team improvises its own path

- Every tool carries its own rules, with no shared standard.
- Cost stays invisible until the invoice arrives.
- No record of what the model did or which data it used.
- Process knowledge is lost with every conversation.
- Vendor lock-in: changing tools means starting over.
- No upgrade path: every new version becomes a project.

Where each part lives, and why it matters

Operation is local; exposure is optional and reversible.

Layer	Residence	Design reason
Orchestration and agents	Local, on the team machine	Process control does not depend on an external service
Operational database (Nexus)	Local SQLite in WAL	Queries without network latency, with programmable backup
Memory and knowledge	Local, with optional sync	Organisational context does not leave the machine by default
Models	Provider or local	Failover chain: provider, then Ollama, then LM Studio
Applications	Local or cloud depending on the case	Exposed by URL when needed, without changing the logic
Audit and traces	Versioned local files	Can be archived, shipped to a SIEM or retained by policy

ECONOMICS

The cost of AI, measured and bounded

The router picks the model by cost, latency and load. Budgets cut before spend becomes a surprise.

10M

Tokens per day

Global budget with a soft cut at 70 percent

3M

Tokens per session

Ceiling per working session

100k

Tokens per agent

Each role carries its own limit

3

Router tiers

cheap, balanced and premium, with chained failover

ADOPTION

How to get in without breaking anything

Four steps, each reversible and each with its own success criterion.

01

Audit

Review the tools, skills and permissions already installed. The audit does not guarantee execution, but it does reveal the real surface.

02

Pilot

One area, one process, one agent. With a token budget and tracing live from day one.

03

Extend

Promote what worked into formal roles, with permissions and limits per role.

04

Govern

Gates, backups, audit and stack upgrades become part of the operating routine.

GentleVanguard

The next step is a diagnostic, not a contract

We propose a working session to audit how AI sits in your organisation today and which part of this platform can be adopted without friction. No cost and no implementation commitment.

Request the stack diagnostic

See the full architecture

WhatsApp: +54 9 264 545-2221

Email: gentlevanguard@gmail.com

Emmanuel Ortiz Gutierrez - Software Architect, AI Integration Engineer

The stack is open source. Internal adoption happens on top of the repository and your own company policies.